

b-secure Backend — Implementation Wiki

Stack: Django 6.0.5 · DRF · Django Channels 4.x · Celery 5 · PostgreSQL/PostGIS · Redis
Python: 3.14.2 · **Venv:** bsecure-backend/venv/
Test DB: SQLite + InMemoryChannelLayer
(DJANGO_SETTINGS_MODULE=config.settings.test)
Test count: 155 / 155 passing

Table of Contents

1. [Project Structure](#)
 2. [Configuration](#)
 3. [Shared Utilities](#)
 4. [Phase 1 — Project Scaffold](#)
 5. [Phase 2 — Shared Utilities](#)
 6. [Phase 3 — Django App Models](#)
 7. [Phase 4 — Authentication App](#)
 8. [Phase 5a — Users App](#)
 9. [Phase 5b — Guards App](#)
 10. [Phase 5c — Bookings App](#)
 11. [Phase 5d — Payments App](#)
 12. [Phase 5e — Notifications, SOS, Reviews, Admin Panel](#)
 13. [Phase 6 — WebSocket Consumers](#)
 14. [Phase 7 — Celery Tasks](#)
 15. [API Endpoint Reference](#)
 16. [Key Design Decisions & Gotchas](#)
 17. [Running Tests](#)
-

1. Project Structure

```
bsecure-backend/  
├─ celery_app.py           # Celery application entry  
point  
├─ manage.py  
└─ pytest.ini              # asyncio_mode=auto,
```

```

DJANGO_SETTINGS_MODULE=config.settings.test
├── config/
│   ├── asgi.py                                # ProtocolTypeRouter +
JWTAuthMiddlewareStack
│   ├── wsgi.py
│   ├── urls.py                                # Root URL conf
│   └── settings/
│       ├── base.py
│       ├── development.py
│       ├── production.py
│       └── test.py                            # SQLite, InMemoryChannelLayer,
CELERY_TASK_ALWAYS_EAGER
├── utils/
│   ├── models.py                            # TimeStampedModel (UUID pk,
created_at, updated_at)
│   ├── permissions.py                       # IsVerifiedUser, IsGuard,
IsAdminUser
│   ├── helpers.py                           # mask_phone_number, haversine,
calculate_booking_price
│   ├── exceptions.py                        # custom_exception_handler,
InsufficientBalanceError, etc.
│   ├── pagination.py                       # StandardResultsPagination
│   └── validators.py                       # validate_phone_number,
validate_pincode
│   ├── mixins.py
│   ├── storage.py
│   └── ws_middleware.py                    # JWTAuthMiddleware (token in
query string)
├── apps/
│   ├── authentication/                     # OTP login, JWT, token
blacklist
│   ├── users/                              # UserProfile, Address,
EmergencyContact
│   ├── guards/                             # GuardProfile, GuardDocument,
Availability
│   ├── bookings/                          # Booking FSM, GuardCheckIn,
BookingBroadcast
│   ├── payments/                          # Wallet, Transaction,
PaymentOrder, Payout
│   ├── notifications/                     # NotificationLog,
NotificationPreference
│   ├── sos/                              # SOSAlert, Incident,
EmergencyContactAlert
│   ├── reviews/                          # Review
│   └── tracking/                          # LocationSnapshot,
TrackingConsumer
│   ├── admin_panel/                      # Admin views +
AdminDashboardConsumer
│   ├── analytics/
│   └── core/

```

2. Configuration

Settings files

File	Purpose
base.py	Shared settings: DRF, SimpleJWT, Channels, Celery queues, SPECTACULAR
development.py	Extends base; DEBUG=True, debug_toolbar
production.py	Extends base; WhiteNoise, Sentry, strict ALLOWED_HOSTS
test.py	SQLite, InMemoryChannelLayer, CELERY_TASK_ALWAYS_EAGER=True

Key settings

```
AUTH_USER_MODEL = "users.UserProfile"
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"  # UUID PKs
                                                         set explicitly on TimeStampedModel

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels.layers.InMemoryChannelLayer",  # test
        # BACKEND: channels_redis.core.RedisChannelLayer      #
        # production
    }
}

CELERY_TASK_ALWAYS_EAGER = True  # synchronous in tests
CELERY_BROKER_URL = "memory://"  # test
```

ASGI routing (config/asgi.py)

```
application = ProtocolTypeRouter({
    "http": get_asgi_application(),
    "websocket": AllowedHostsOriginValidator(
        JWTAuthMiddlewareStack(
            URLRouter(
                tracking_websocket_urlpatterns +  # /ws/tracking/
                {id}/
                sos_websocket_urlpatterns +      # /ws/sos/
                feed/
                admin_websocket_urlpatterns      # /ws/admin/
                dashboard/
            )
        )
    )
})
```

```
    ),  
})
```

3. Shared Utilities

utils/models.py — TimeStampedModel

Base model used by every app model:

```
class TimeStampedModel(models.Model):  
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,  
        editable=False)  
    created_at = models.DateTimeField(auto_now_add=True)  
    updated_at = models.DateTimeField(auto_now=True)  
  
    class Meta:  
        abstract = True
```

utils/permissions.py

Class	Logic
IsVerifiedUser	user.is_authenticated and not user.is_suspended
IsGuard	is_authenticated + has guard_profile + verification_status == "ACTIVE"
IsAdminUser	is_authenticated + is_staff

utils/helpers.py

Function	Notes
mask_phone_number(phone)	phone[:3] + '*****' + phone[-4:] (always 5 stars)
haversine_distance(lat1, lng1, lat2, lng2)	Returns km
calculate_booking_price(rate, hours, fee_pct, tax_pct)	Returns (total, platform_fee, guard_earnings)
generate_otp(length)	Zero-padded random numeric string
hash_otp(otp) / verify_otp(raw, hashed)	bcrypt

utils/ws_middleware.py — JWTAuthMiddleware

Extracts ?token=<jwt> from WebSocket query string, validates with SimpleJWT, sets scope["user"]. Falls back to AnonymousUser on any failure.

utils/exceptions.py — custom_exception_handler

All error responses follow:

```
{"error": {"code": "...", "message": "...", "details": {}}}
```

Custom exceptions: InsufficientBalanceError, NoGuardsAvailableError, OTPExpiredError, OTPInvalidError, OTPRateLimitError.

4. Phase 1 — Project Scaffold

- Django project created with `django-admin startproject`
 - All apps created: authentication, users, guards, bookings, payments, notifications, sos, reviews, tracking, admin_panel, analytics, core
 - requirements.txt with all dependencies installed in venv
 - pytest.ini configured with `asyncio_mode = auto`
-

5. Phase 2 — Shared Utilities

All files in utils/ written and tested: - models.py, permissions.py, helpers.py, exceptions.py, pagination.py, validators.py, mixins.py, storage.py, ws_middleware.py

6. Phase 3 — Django App Models

All models written and migrated. Key notes:

PostGIS compatibility

PointField is conditionally imported with fallback to TextField for SQLite test env:

```
# apps/guards/models.py, apps/tracking/models.py
try:
    from django.contrib.gis.db import models as gis_models
    _PointField = gis_models.PointField
```

```

except Exception:
    _PointField = None

current_location = (
    _PointField(geography=True, null=True, blank=True)
    if _PointField is not None
    else models.TextField(null=True, blank=True)
)

```

Booking FSM (apps/bookings/models.py)

Uses django-fsm. The state field is named **status** (not state).

REQUESTED → BROADCAST → ACCEPTED → EN_ROUTE → ARRIVED → ACTIVE → COMPLETED

DISPUTED

↘ EXPIRED

↘ CANCELLED (from most states)

Transitions: start_broadcast, accept, en_route, arrive, verify_start_otp, complete, cancel, expire, dispute.

Wallet signal (apps/users/signals.py)

Auto-creates Wallet and NotificationPreference on user creation. In tests always use `Wallet.objects.get_or_create(user=user)` to avoid UNIQUE constraint errors.

Transaction model

Requires `balance_before` and `balance_after` fields (non-null). Always provide these when creating transactions.

7. Phase 4 — Authentication App

Files: models.py, services.py, serializers.py, views.py, throttles.py, urls.py

Tests: test_phases_1_to_4.py — 47/47 pass

OTP Flow

POST /api/auth/send-otp/ → OTPService.send_otp(phone) → creates OTPToken, returns masked phone
 POST /api/auth/verify-otp/ → OTPService.verify_otp(phone, code) → returns {access, refresh, is_new_user}

POST /api/auth/logout/ → blacklists refresh token
POST /api/auth/token/refresh/ → SimpleJWT token refresh

OTPToken model

- 6-digit code, hashed with bcrypt
- Expires after settings.OTP_EXPIRY_SECONDS (default 300)
- Locked after settings.OTP_MAX_ATTEMPTS (default 3) failed attempts
- Previous tokens invalidated on new send

Throttle rates

```
# apps/authentication/throttles.py
# Must use DRF suffix format: s, m, h, d
"5/m" # OTP send
"10/m" # OTP verify
```

8. Phase 5a — Users App

Files: serializers.py, services.py, views.py, urls.py

Tests: test_users.py — 24/24 pass

Endpoints

Method	URL	Description
GET/PATCH/PUT	/api/users/me/	Get/update own profile
GET/POST	/api/users/me/ addresses/	List/create addresses (max 10)
PUT/DELETE	/api/users/me/ addresses/{id}/	Update/delete address
GET/POST	/api/users/me/ emergency-contacts/	List/create contacts (max 5)
DELETE	/api/users/me/ emergency-contacts/ {id}/	Delete contact
POST	/api/users/me/fcm- token/	Update FCM push token
DELETE	/api/users/me/	Request account deletion
GET/PUT		Notification settings

Method	URL	Description
	/api/users/me/ notification- preferences/	

UserService

Business logic in `apps/users/services.py`: - `get_or_create_profile`, `update_profile`, `update_fcm_token` - `create_address`, `update_address`, `delete_address` - `create_emergency_contact`, `delete_emergency_contact` - `request_account_deletion` (sets `deletion_requested=True`, schedules Celery task)

9. Phase 5b — Guards App

Files: `serializers.py`, `services.py`, `views.py`, `urls.py`

Tests: `test_guards.py` — 18/18 pass

Endpoints

Method	URL	Description
GET/PATCH	/api/guards/me/	Get/update own guard profile
POST	/api/guards/me/ online/	Go online (requires <code>verification_status == "ACTIVE"</code>)
POST	/api/guards/me/ offline/	Go offline
GET/POST	/api/guards/me/ documents/	List/upload documents
GET/POST/PUT	/api/guards/me/ availability/	Get/set weekly availability
GET	/api/guards/{id}/	Public guard profile

GuardService

- `get_guard_profile`, `update_guard_profile`
- `set_online_status` — raises `PermissionDenied` if `verification_status != "ACTIVE"`
- `upload_document`, `set_availability`

10. Phase 5c — Bookings App

Files: serializers.py, services.py, views.py, urls.py

Tests: test_bookings.py — 17/17 pass

Endpoints

Method	URL	Description
POST	/api/bookings/	Create booking
GET	/api/bookings/	List user's bookings
GET	/api/bookings/{id}/	Booking detail
POST	/api/bookings/{id}/cancel/	Cancel booking
POST	/api/bookings/{id}/otp/start/generate/	Generate session-start OTP
POST	/api/bookings/{id}/otp/start/verify/	Verify session-start OTP (→ ACTIVE)
POST	/api/bookings/{id}/otp/end/generate/	Generate session-end OTP
POST	/api/bookings/{id}/otp/end/verify/	Verify session-end OTP (→ COMPLETED)
POST	/api/bookings/{id}/en-route/	Guard marks en-route
POST	/api/bookings/{id}/arrived/	Guard marks arrived
GET	/api/bookings/active/	Get user's current active booking

BookingService

Key notes: - Uses `Wallet.objects.get(user=user)` — **never** `user.wallet` (cached reverse relation) - `broadcast_status_change(booking)` — sends `session_status_update` to WS group `session_{id}` - Platform fee: 15% of `total_amount`

11. Phase 5d — Payments App

Files: serializers.py, services.py, views.py, urls.py

Tests: test_payments.py — 10/10 pass

Endpoints

Method	URL	Description
GET	/api/payments/wallet/	Get wallet balance
POST	/api/payments/topup/initiate/	Create Razorpay order for wallet topup
POST	/api/payments/topup/confirm/	Confirm payment, credit wallet
GET	/api/payments/transactions/	Transaction history
POST	/api/payments/webhooks/razorpay/	Razorpay webhook
POST	/api/payments/webhooks/stripe/	Stripe webhook

Models

- **Wallet** — user (OneToOne), balance, is_frozen
- **Transaction** — wallet, transaction_type, amount, balance_before, balance_after (both required), status, booking (FK nullable)
- **PaymentOrder** — user, order_type, amount, gateway, gateway_order_id, gateway_response (JSON), status
- **Payout** — guard, amount, status, processed_at

Transaction types

TOPUP, BOOKING_DEBIT, BOOKING_REFUND, TOPUP (for guard earnings — no “CREDIT” type)

12. Phase 5e — Notifications, SOS, Reviews, Admin Panel

Tests: test_phase5e.py — 21/21 pass

Notifications

Method	URL	Description
GET	/api/notifications/	List notifications
GET	/api/notifications/unread-count/	Unread count

Method	URL	Description
POST	/api/notifications/{id}/read/	Mark one read
POST	/api/notifications/read-all/	Mark all read

SOS

Method	URL	Description
POST	/api/sos/trigger/	Trigger SOS alert
GET	/api/sos/alerts/	List user's SOS alerts
GET	/api/sos/alerts/{id}/	SOS detail
GET/POST	/api/incidents/	List/file incident
GET	/api/incidents/{id}/	Incident detail

Reviews

Method	URL	Description
POST	/api/reviews/	Submit review (booking must be COMPLETED)
GET	/api/reviews/	List reviews for requesting user

Review signal auto-updates `GuardProfile.average_rating`.

Admin Panel

Method	URL	Description
GET	/api/admin/dashboard/stats/	Live KPI stats
GET	/api/admin/users/	User list
POST	/api/admin/users/{id}/suspend/	Suspend user
POST	/api/admin/users/{id}/unsuspend/	Unsuspend user
POST	/api/admin/users/{id}/credit-wallet/	Credit wallet
GET	/api/admin/guards/	Guard list
POST	/api/admin/guards/{id}/approve/	Approve guard
GET	/api/admin/sos/alerts/	SOS alert list

13. Phase 6 — WebSocket Consumers

Tests: apps/tracking/tests/test_websockets.py — 18/18 pass

TrackingConsumer — apps/tracking/consumers.py

URL: /ws/tracking/{booking_id}/

Groups: session_{booking_id}, admin_live_map

Roles: - **Guard** (assigned to booking): sends location_update and ping messages - **User** (booking owner): read-only, receives guard_location and session_status_change - **Admin** (is_staff=True): joins both session_* and admin_live_map groups

Inbound messages (guard only):

type	Payload	Action
location_update	{lat, lng, accuracy, speed, bearing}	Save LocationSnapshot, update GuardProfile.last_location_update, broadcast to group
ping	{}	Responds with pong

Outbound messages:

type	Trigger
session_state	On connect (current booking status)
guard_location	On each guard location update
session_status_change	When BookingService.broadcast_status_change() called
error	When non-guard tries to send
pong	Response to ping

Close codes:

Code	Reason
4001	Anonymous / not authenticated
4003	Not a participant in this booking
4004	Booking not found

LocationSnapshot field compatibility:

```
# _save_location_snapshot uses location field (not latitude/
    longitude)
try:
    from django.contrib.gis.geos import Point
    location_value = Point(float{lng}, float{lat}, srid=4326) #
        PostGIS
except Exception:
    location_value = f"{{lat}},{{lng}}" # SQLite fallback (TextField)
```

SOSFeedConsumer — apps/sos/consumers.py

URL: /ws/sos/feed/
Group: admin_sos_feed

Admin-only. Rejects non-staff with code 4001.

Inbound messages:

type	Action
acknowledge_sos	{sos_id} — updates SOSAlert to ACKNOWLEDGED in DB

Outbound messages (from group broadcasts):

type	Handler method	Trigger
sos_alert	sos_alert(event)	SOSService._broadcast_to_admins(sos)
sos_status_update	sos_status_update(event)	SOSService._broadcast_status_update(sos)

AdminDashboardConsumer — apps/admin_panel/consumers.py

URL: /ws/admin/dashboard/
Group: admin_dashboard

Admin-only. Rejects non-staff with code 4001.

On connect, immediately sends:

```
{"type": "initial_stats", "data": {"active_sessions": N,
    "guards_online": N, "open_sos_alerts": N}}
```

Outbound messages:

type	Handler	Trigger
dashboard_update	dashboard_update(event)	

type	Handler	Trigger
		push_live_dashboard_stats Celery task (every 30s)

Static broadcast helper:

```
AdminDashboardConsumer.broadcast_stats_update(stats_dict)
```

Routing files

```
# apps/tracking/routing.py
tracking_websocket_urlpatterns = [
    re_path(r"^ws/tracking/(?P<booking_id>[0-9a-f-]{36})/$",
            TrackingConsumer.as_asgi()),
]

# apps/sos/routing.py
sos_websocket_urlpatterns = [
    re_path(r"^ws/sos/feed/$", SOSFeedConsumer.as_asgi()),
]

# apps/admin_panel/routing.py
admin_websocket_urlpatterns = [
    re_path(r"^ws/admin/dashboard/$",
            AdminDashboardConsumer.as_asgi()),
]
```

SOSService — apps/sos/services.py

Full service layer for SOS operations:

```
SOSService.trigger_sos(user, trigger_method, latitude, longitude,
                       booking=None)
    # 1. Creates SOSAlert synchronously
    # 2. Broadcasts to admin_sos_feed WS group
    # 3. Queues notify_emergency_contacts (high_priority)
    # 4. Queues notify_guard_of_user_sos if booking ACTIVE
        (high_priority)
    # 5. Schedules schedule_sos_escalation in 300s (high_priority)

SOSService.acknowledge_sos(sos_id, admin_user)
SOSService.resolve_sos(sos_id, admin_user, notes,
                       is_false_alarm=False)
```

14. Phase 7 — Celery Tasks

All tasks are auto-discovered via `celery_app.autodiscover_tasks()`.

Queue Architecture

Queue	Workers	Tasks
high_priority	4	OTP, SOS, booking broadcast, emergency notifications
default	4	Standard notifications, payment processing, check-in monitoring
low_priority	2	PDF, analytics aggregation, cleanup
scheduled	1 (beat)	Nightly payouts, token cleanup, document expiry

Celery Beat Schedule

Task	Schedule	Queue
check_guard_offline_sessions	Every 5 min	default
monitor_session_checkins	Every 15 min	default
push_live_dashboard_stats	Every 30 sec	default
expire_stale_broadcasts	Every 5 min	default
send_checkin_reminders	Every 5 min	default
aggregate_daily_stats	Daily 23:55	low_priority
cleanup_expired_tokens	Daily 00:00	scheduled
check_document_expiry	Daily 08:00	scheduled
process_weekly_payouts	Friday 22:00	scheduled
cleanup_old_location_snapshots	Sunday 00:00	low_priority

Task Reference

`apps/authentication/tasks.py`

Task	Description
cleanup_expired_tokens	

Task	Description
	Deletes OTPTokens > 24h old and expired JWT blacklist entries

apps/bookings/tasks.py

Task	Description
<code>broadcast_booking_request(booking_id, radius_km=5)</code>	Finds nearby guards (PostGIS or fallback), sends push notifications, schedules expiry. Expands radius to 10km if no guards found.
<code>expire_unaccepted_booking(booking_id)</code>	Expires BROADCAST booking after 5 min
<code>expire_booking(booking_id)</code>	Expires booking when max radius reached with no guards
<code>expire_stale_broadcasts</code>	Beat task: expires all stale BROADCAST bookings past timeout
<code>send_checkin_reminders</code>	Beat task: sends push to guards whose check-in is due within 15 min

apps/guards/tasks.py

Task	Description
<code>check_document_expiry</code>	Finds docs expiring within 30 days, sends push notification, marks <code>expiry_reminder_sent=True</code>

apps/analytics/tasks.py

Task	Description
<code>aggregate_daily_stats</code>	Aggregates previous day's bookings/revenue/users/SOS into DailyStats
<code>cleanup_old_location_snapshots</code>	Deletes LocationSnapshot records older than 90 days

apps/sos/tasks.py

Task	Description
<code>notify_emergency_contacts(sos_alert_id)</code>	SMS all emergency contacts with GPS link, creates EmergencyContactAlert records
<code>schedule_sos_escalation(sos_id)</code>	At +5 min: if still TRIGGERED, SMS supervisor phones, schedules second escalation
<code>schedule_second_escalation(sos_id)</code>	At +15 min: if still unresolved, emails on-call engineer
<code>monitor_session_checkins</code>	Beat task: checks DAILY/WEEKLY/MONTHLY sessions for missed check-ins; escalates to SOS after 30 min
<code>check_guard_offline_sessions (named dead_mans_switch_check)</code>	Beat task: triggers SOS when guard goes silent during ACTIVE session

apps/notifications/tasks.py

Task	Description
<code>send_push_notification(user_id, title, body, data)</code>	Logs NotificationLog, sends via FCM in production, console in test/dev
<code>send_sms(phone_number, message)</code>	Sends via Twilio in production, console in test/dev
<code>send_email(to_email, subject, body)</code>	Sends via Django email backend
<code>notify_guard_of_user_sos(booking_id)</code>	Push notification to assigned guard on user SOS

apps/payments/tasks.py

Task	Description
process_pending_payouts	Processes all PENDING payouts with select_for_update to prevent double-processing
process_weekly_payouts	Creates Payout records for guards with balance $\geq$ ₹100, triggers process_pending_payouts
process_razorpay_event(event)	Handles Razorpay webhook events (payment.captured)
process_stripe_event(event)	Handles Stripe webhook events (payment_intent.succeeded)

apps/admin_panel/tasks.py

Task	Description
push_live_dashboard_stats	Collects live KPIs, calls AdminDashboardConsumer.broadcast_stats_update()

15. API Endpoint Reference

Base URL: /api/

App	Prefix	Mounted in config/urls.py
authentication	/api/auth/	apps.authentication.urls
users	/api/users/	apps.users.urls
guards	/api/guards/	apps.guards.urls
bookings	/api/bookings/	apps.bookings.urls
payments	/api/payments/	apps.payments.urls
notifications	/api/notifications/	apps.notifications.urls
sos	/api/sos/	apps.sos.urls
incidents	/api/incidents/	apps.sos.incident_urls
reviews	/api/reviews/	apps.reviews.urls
admin_panel	/api/admin/	apps.admin_panel.urls
core	/api/	apps.core.urls
schema	/api/schema/	drf-spectacular

App	Prefix	Mounted in config/urls.py
docs	/api/docs/	SpectacularSwaggerView

Standard response envelope

Success:

```
{"data": {...}}
```

Error:

```
{"error": {"code": "ERROR_CODE", "message": "Human readable  
message", "details": {}}}
```

16. Key Design Decisions & Gotchas

1. DEFAULT_AUTO_FIELD must not be UUIDField

Django does not support UUIDField as DEFAULT_AUTO_FIELD. Use BigAutoField. UUID PKs are applied explicitly on TimeStampedModel.

2. SpectacularSwaggerUIView → SpectacularSwaggerView

The view was renamed in drf-spectacular 0.29.0.

3. PostGIS PointField conditional import

Both apps/guards/models.py and apps/tracking/models.py use try/except to fall back to TextField when PostGIS is not available (SQLite test env).

4. debug_toolbar URL guard

```
if "debug_toolbar" in settings.INSTALLED_APPS:
    import debug_toolbar
    urlpatterns += [path("__debug__/",
        include(debug_toolbar.urls))]
```

5. Throttle rate format

DRF only accepts s, m, h, d suffixes. Custom formats like 5/10min cause KeyError. Use "5/m" and "10/m".

6. mask_phone_number format

Always produces exactly 5 asterisks: `phone[:3] + '*****' + phone[-4:]`

7. IsGuard permission

Requires `verification_status == "ACTIVE"`. Test fixtures must set this explicitly.

8. Booking FSM field name

The FSMField is named `status`, not `state`. Filters must use `status=`.

9. Wallet reverse relation

`user.wallet` is cached after signal creation. Always use `Wallet.objects.get(user=user)` in service layer for fresh data.

10. Wallet signal in tests

Wallet is auto-created by signal on user creation. Use `get_or_create` in test fixtures:

```
wallet, _ = Wallet.objects.get_or_create(user=user)
```

11. PaymentOrder model

No `gateway_payment_id` field. Use `gateway_response` (JSONField) to store gateway data.

12. Transaction required fields

`balance_before` and `balance_after` are non-null. Must always be provided:

```
Transaction.objects.create(
    ...,
    balance_before=wallet.balance,
    balance_after=wallet.balance + amount,
)
```

13. transaction_type choices

No "CREDIT" type. For guard earnings use "TOPUP". Available types: TOPUP, BOOKING_DEBIT, BOOKING_REFUND

14. pytest fixture naming

The name `client` conflicts with `pytest-django`'s built-in `client` fixture. Use `api_client` instead.

15. LocationSnapshot.location field

The model uses a single location field (PostGIS PointField or TextField), not separate latitude/longitude fields. The consumer handles both:

```
try:
    from django.contrib.gis.geos import Point
    location_value = Point(float{lng}, float{lat}, srid=4326)
except Exception:
    location_value = f"{{lat}},{{lng}}"
```

16. WebSocket testing

- Install `daphne` for `channels.testing.WebsocketCommunicator` to work
 - Inject user directly into `communicator.scope["user"]` (no JWT in tests)
 - Use `@pytest.mark.django_db(transaction=True)` for async WS tests
 - `asyncio_mode = auto` in `pytest.ini` handles `@pytest.mark.asyncio` automatically
-

17. Running Tests

```
# Activate venv
source venv/bin/activate

# Run all tests
pytest --tb=short

# Run specific app
pytest apps/tracking/tests/test_websockets.py -v
pytest apps/bookings/tests/test_bookings.py -v

# Run with coverage
pytest --cov=apps --cov-report=html

# Expected: 155 passed, 2 warnings
```

Test settings

- **DB:** SQLite (no PostGIS required)
- **Channel Layer:** `InMemoryChannelLayer`
- **Celery:** `CELERY_TASK_ALWAYS_EAGER = True` (synchronous execution)

- **SMS/Push backend:** "console" (logged, not sent)
- **Email backend:** locmem (captured in `django.core.mail.outbox`)